

Листинг программы

Вариант 4. Решение систем линейных алгебраических уравнений. Листинг исходных файлов проекта в порядке сборки.

Program.cs

```
1 using System;
2 using LabsUtil;
3
4 // Лабораторная работа №1. Методы решения систем линейных алгебраических уравнений.
5 // Задание 1 — метод Гаусса с выбором главных элементов.
6 // Задание 2 (чётный вариант) — метод Зейделя с точностью 10-5
7 class Program
8 {
9     private const double Epsilon = 1e-5;
10
11     private static void Main(string[] args)
12     {
13         Console.WriteLine("Лабораторная работа №1. Методы решения систем линейных алгебраических уравнений");
14         Console.WriteLine("Вариант 4");
15         Console.WriteLine();
16
17         var system = ReadSystem();
18         Console.WriteLine();
19         system.Print("Исходная система:");
20         Console.WriteLine();
21
22         Console.WriteLine("ЗАДАНИЕ 1. Решение методом Гаусса с выбором главных элементов");
23         Console.WriteLine();
24         var gaussSolution = GaussSolver.Solve(system);
25         if (gaussSolution == null)
26         {
27             Console.WriteLine("Метод Гаусса не дал единственного решения.");
28         }
29         else
30         {
31             PrintSolution("Решение методом Гаусса:", gaussSolution);
32             system.PrintResidual("Проверка подстановкой в исходную систему:", gaussSolution);
33         }
34         Console.WriteLine();
35
36         Console.WriteLine("ЗАДАНИЕ 2. Решение методом Зейделя с точностью 10-5");
37         Console.WriteLine();
38         var iterationSystem = IterationForm.Build(system);
39         double[]? zeidelSolution = null;
40         if (iterationSystem != null)
41         {
42             Console.WriteLine();
43             zeidelSolution = ZeidelSolver.Solve(iterationSystem, Epsilon);
44             if (zeidelSolution == null)
45             {
46                 Console.WriteLine("Метод Зейделя не сошёлся.");
47             }
48             else
49             {
50                 PrintSolution("Решение методом Зейделя:", zeidelSolution);
51                 system.PrintResidual("Проверка подстановкой в исходную систему:", zeidelSolution);
52             }
53         }
54         Console.WriteLine();
55
56         if (gaussSolution == null || zeidelSolution == null)
57         {
58             return;
59         }
60         var difference = 0.0;
```

```

61for (var i = 0; i < system.Size; i++)
62{
63difference = Math.Max(difference, Math.Abs(gaussSolution[i] - zeidelSolution[i]));
64}
65Console.WriteLine("Сравнение методов: максимальное расхождение решений = "
66+ NumberFormat.Format(difference, 10));
67}
68
69// Универсальный ввод данных: система варианта 4 или произвольная система
70private static LinearSystem ReadSystem()
71{
72Console.WriteLine("1 : система варианта 4");
73Console.WriteLine("2 : ввести свою систему");
74var choice = PstuUtil.TryReadT<int>("выберите исходную систему: ");
75if (choice == 1)
76{
77return LinearSystem.CreateVariant4();
78}
79
80var size = PstuUtil.TryReadT<int>("число уравнений n: ", 1, 6);
81var coefficients = new double[size, size];
82var rightParts = new double[size];
83for (var row = 0; row < size; row++)
84{
85for (var column = 0; column < size; column++)
86{
87coefficients[row, column] = PstuUtil.TryReadT<double>("a" + (row + 1) + (column + 1) + " = ");
88}
89rightParts[row] = PstuUtil.TryReadT<double>("b" + (row + 1) + " = ");
90}
91return new LinearSystem(coefficients, rightParts);
92}
93
94private static void PrintSolution(string title, double[] solution)
95{
96Console.WriteLine();
97Console.WriteLine(title);
98Console.WriteLine(" " + NumberFormat.Cells(solution, 8));
99}
100}

```

Program.cs — строк: 100

LinearSystem.cs

```

1using System;
2using System.Text;
3
4// Система линейных алгебраических уравнений Ax = b:
5// коэффициенты хранятся в двумерном массиве, свободные члены — в одномерном
6class LinearSystem
7{
8// Число, меньше которого коэффициент считается нулём (защита от деления на ноль)
9public const double Tiny = 1e-12;
10
11private const int Digits = 4;
12
13private readonly double[,] _coefficients;
14private readonly double[] _rightParts;
15
16public LinearSystem(double[,] coefficients, double[] rightParts)
17{
18_coefficients = coefficients;
19_rightParts = rightParts;
20}
21
22public int Size => _rightParts.Length;
23
24public double GetCoefficient(int row, int column) => _coefficients[row, column];

```

```

25
26 public double GetRightPart(int row) => _rightParts[row];
27
28 // Вариант 4 из лабораторной работы
29 public static LinearSystem CreateVariant4()
30 {
31     var coefficients = new double[,]
32     {
33         { 9.1, 5.6, 7.8 },
34         { 3.8, 5.1, 2.8 },
35         { 4.1, 5.7, 1.2 }
36     };
37     var rightParts = new double[] { 9.8, 6.7, 5.8 };
38     return new LinearSystem(coefficients, rightParts);
39 }
40
41 // Собирает систему из расширенных строк (n коэффициентов и свободный член)
42 public static LinearSystem FromRows(double[][] extendedRows)
43 {
44     var size = extendedRows.Length;
45     var coefficients = new double[size, size];
46     var rightParts = new double[size];
47     for (var row = 0; row < size; row++)
48     {
49         for (var column = 0; column < size; column++)
50         {
51             coefficients[row, column] = extendedRows[row][column];
52         }
53         rightParts[row] = extendedRows[row][size];
54     }
55     return new LinearSystem(coefficients, rightParts);
56 }
57
58 public double[] GetExtendedRow(int row)
59 {
60     var extendedRow = new double[Size + 1];
61     for (var column = 0; column < Size; column++)
62     {
63         extendedRow[column] = _coefficients[row, column];
64     }
65     extendedRow[Size] = _rightParts[row];
66     return extendedRow;
67 }
68
69 // Линейная комбинация уравнений системы с целыми коэффициентами:
70 // новая строка = c1·уравнение1 + c2·уравнение2 + ... + cn·уравнениеn
71 public double[] GetCombinedRow(int[] coefficients)
72 {
73     var extendedRow = new double[Size + 1];
74     for (var k = 0; k < Size; k++)
75     {
76         var source = GetExtendedRow(k);
77         for (var column = 0; column <= Size; column++)
78         {
79             extendedRow[column] += coefficients[k] * source[column];
80         }
81     }
82     return extendedRow;
83 }
84
85 // Отношение суммы модулей внедиагональных элементов строки к её диагональному
86 // элементу. Менее единицы означает строгое диагональное преобладание
87 public static double GetDominanceRatio(double[] extendedRow, int diagonalIndex)
88 {
89     var size = extendedRow.Length - 1;
90     var diagonal = Math.Abs(extendedRow[diagonalIndex]);
91     if (diagonal < Tiny)
92     {
93         return double.PositiveInfinity;
94     }
95 }

```

```

94}
95var offDiagonal = 0.0;
96for (var column = 0; column < size; column++)
97{
98if (column == diagonalIndex)
99{
100continue;
101}
102offDiagonal += Math.Abs(extendedRow[column]);
103}
104return offDiagonal / diagonal;
105}
106
107public bool IsStrictlyDominant()
108{
109for (var row = 0; row < Size; row++)
110{
111if (GetDominanceRatio(GetExtendedRow(row), row) >= 1)
112{
113return false;
114}
115}
116return true;
117}
118
119public void Print(string title)
120{
121Console.WriteLine(title);
122for (var row = 0; row < Size; row++)
123{
124Console.WriteLine(" " + FormatEquation(GetExtendedRow(row)));
125}
126}
127
128// Проверка результата подстановкой в исходную систему: печатает остатки  $r = b - Ax$ 
129public void PrintResidual(string title, double[] solution)
130{
131Console.WriteLine();
132Console.WriteLine(title);
133var norm = 0.0;
134for (var row = 0; row < Size; row++)
135{
136var residual = _rightParts[row];
137for (var column = 0; column < Size; column++)
138{
139residual -= _coefficients[row, column] * solution[column];
140}
141norm = Math.Max(norm, Math.Abs(residual));
142Console.WriteLine(" r" + (row + 1) + " = " + NumberFormat.Format(residual, 10));
143}
144Console.WriteLine(" норма остатка = " + NumberFormat.Format(norm, 10));
145}
146
147private static string FormatEquation(double[] extendedRow)
148{
149var size = extendedRow.Length - 1;
150var text = new StringBuilder();
151for (var column = 0; column < size; column++)
152{
153if (column == 0)
154{
155text.Append(NumberFormat.Format(extendedRow[column], Digits));
156}
157else
158{
159text.Append(extendedRow[column] < 0 ? " - " : " + ");
160text.Append(NumberFormat.Format(Math.Abs(extendedRow[column]), Digits));
161}
162text.Append(" x").Append(column + 1);

```

```

163}
164text.Append(" ").Append(NumberFormat.Format(extendedRow[size], Digits));
165return text.ToString();
166}
167}

```

LinearSystem.cs — строк: 167

GaussSolver.cs

```

1using System;
2
3// Метод Гаусса с выбором главных элементов (метод главных элементов).
4// На каждом шаге выбирается наибольший по модулю элемент оставшегося блока,
5// переносится на главную диагональ перестановкой строк и столбцов,
6// после чего его столбец исключается из остальных строк
7static class GaussSolver
8{
9    private const int Digits = 4;
10    private const int MultiplierDigits = 6;
11    private const int SolutionDigits = 8;
12
13    public static double[]? Solve(LinearSystem system)
14    {
15        var size = system.Size;
16
17        // Расширенная матрица системы: n строк и n + 1 столбцов
18        var extended = new double[size, size + 1];
19        for (var row = 0; row < size; row++)
20        {
21            for (var column = 0; column < size; column++)
22            {
23                extended[row, column] = system.GetCoefficient(row, column);
24            }
25            extended[row, size] = system.GetRightPart(row);
26        }
27
28        // При перестановке столбцов меняется нумерация неизвестных
29        var variableOrder = new int[size];
30        for (var i = 0; i < size; i++)
31        {
32            variableOrder[i] = i;
33        }
34
35        // Прямой ход
36        for (var step = 0; step < size; step++)
37        {
38            FindMainElement(extended, step, out var mainRow, out var mainColumn, out var mainValue);
39
40            // Деление на ноль исключено: если главного элемента нет,
41            // система не имеет единственного решения
42            if (Math.Abs(mainValue) < LinearSystem.Tiny)
43            {
44                ReportNoUniqueSolution(extended, step);
45                return null;
46            }
47
48            Console.WriteLine("Шаг " + (step + 1) + ", Главный элемент a"
49                + (mainRow + 1) + (mainColumn + 1) + " = "
50                + NumberFormat.Format(mainValue, Digits));
51            SwapRows(extended, mainRow, step);
52            SwapColumns(extended, mainColumn, step);
53            (variableOrder[step], variableOrder[mainColumn]) = (variableOrder[mainColumn], variableOrder[step]);
54            if (mainColumn != step)
55            {
56                Console.WriteLine(" столбец " + (mainColumn + 1) + " перенесён на место столбца "
57                    + (step + 1) + ", нумерация неизвестных меняется");
58            }
59        }

```

```

60for (var row = step + 1; row < size; row++)
61{
62var multiplier = extended[row, step] / extended[step, step];
63extended[row, step] = 0;
64for (var column = step + 1; column <= size; column++)
65{
66extended[row, column] -= multiplier * extended[step, column];
67}
68Console.WriteLine(" множитель m" + (row + 1) + " = "
69+ NumberFormat.Format(multiplier, MultiplierDigits));
70}
71
72Console.WriteLine(" приведённая система:");
73Console.WriteLine(NumberFormat.Block(extended, step, step, size, Digits));
74Console.WriteLine();
75}
76
77// Обратный ход
78var current = new double[size];
79for (var row = size - 1; row >= 0; row--)
80{
81if (Math.Abs(extended[row, row]) < LinearSystem.Tiny)
82{
83Console.WriteLine(" Диагональный элемент равен нулю, система не имеет единственного решения.");
84Console.WriteLine(" Деление на ноль не выполняется.");
85return null;
86}
87var value = extended[row, size];
88for (var column = row + 1; column < size; column++)
89{
90value -= extended[row, column] * current[column];
91}
92current[row] = value / extended[row, row];
93Console.WriteLine(" x" + (variableOrder[row] + 1) + " = "
94+ NumberFormat.Format(current[row], SolutionDigits));
95}
96
97// Возвращаем решение в исходной нумерации неизвестных
98var solution = new double[size];
99for (var i = 0; i < size; i++)
100{
101solution[variableOrder[i]] = current[i];
102}
103return solution;
104}
105
106// Наибольший по модулю элемент блока, начинающегося с шага step
107private static void FindMainElement(double[,] extended, int step, out int mainRow, out int mainColumn, out double
    mainValue)
108{
109var size = extended.GetLength(0);
110mainRow = step;
111mainColumn = step;
112mainValue = 0;
113for (var row = step; row < size; row++)
114{
115for (var column = step; column < size; column++)
116{
117if (Math.Abs(extended[row, column]) <= Math.Abs(mainValue))
118{
119continue;
120}
121mainValue = extended[row, column];
122mainRow = row;
123mainColumn = column;
124}
125}
126}
127

```

```

128 private static void SwapRows(double[,] extended, int first, int second)
129 {
130     if (first == second)
131     {
132         return;
133     }
134     var width = extended.GetLength(1);
135     for (var column = 0; column < width; column++)
136     {
137         var value = extended[first, column];
138         extended[first, column] = extended[second, column];
139         extended[second, column] = value;
140     }
141 }
142
143 // Столбец свободных членов не переставляется
144 private static void SwapColumns(double[,] extended, int first, int second)
145 {
146     if (first == second)
147     {
148         return;
149     }
150     var height = extended.GetLength(0);
151     for (var row = 0; row < height; row++)
152     {
153         var value = extended[row, first];
154         extended[row, first] = extended[row, second];
155         extended[row, second] = value;
156     }
157 }
158
159 // Все элементы столбца ниже диагонального нулевые, поэтому определитель
160 // равен нулю. Система либо несовместна, либо имеет бесконечно много решений
161 private static void ReportNoUniqueSolution(double[,] extended, int step)
162 {
163     var size = extended.GetLength(0);
164     var isConsistent = true;
165     for (var row = step; row < size; row++)
166     {
167         var isZero = true;
168         for (var column = step; column < size; column++)
169         {
170             if (Math.Abs(extended[row, column]) <= LinearSystem.Tiny)
171             {
172                 continue;
173             }
174             isZero = false;
175             break;
176         }
177         if (isZero && Math.Abs(extended[row, size]) > LinearSystem.Tiny)
178         {
179             isConsistent = false;
180         }
181     }
182
183     Console.WriteLine("Главный элемент не найден: все элементы оставшегося блока равны нулю.");
184     if (isConsistent)
185     {
186         Console.WriteLine("Определитель системы равен нулю, система имеет бесконечно много решений.");
187     }
188     else
189     {
190         Console.WriteLine("Определитель системы равен нулю и система несовместна, решения нет.");
191     }
192     Console.WriteLine("Деление на ноль не выполняется, аварийное завершение исключено.");
193 }
194

```

GaussSolver.cs — строк: 194

IterationForm.cs

```
1using System;
2using System.Text;
3
4// Приведение системы к виду, удобному для итераций, и проверка условия сходимости.
5// Система  $Ax = b$  приводится к виду  $x = Cx + d$ , где  $C = I - D^{-1} \cdot A$ ,  $d = D^{-1} \cdot A^{-1} \cdot b$ 
6// (на практике  $d_i = b_i / a_{ii}$ ). Для метода Зейделя достаточно, чтобы
7//  $\|C\| < 1$  по какой-либо норме матрицы. Проверяются нормы, подчинённые
8// октаэдрической ( $\|C\|_1 = \max$  по столбцам) и кубической ( $\|C\|_\infty = \max$  по строкам)
9// нормам векторов
10static class IterationForm
11{
12// Коэффициенты подбираемых комбинаций уравнений
13private const int MaxCombinationCoefficient = 2;
14
15// При большей размерности перебор комбинаций слишком велик
16private const int MaxSizeForCombinationSearch = 5;
17
18private const int Digits = 4;
19
20// Возвращает систему, пригодную для итераций, либо null,
21// если привести систему не удалось
22public static LinearSystem? Build(LinearSystem source)
23{
24Console.WriteLine("Проверка условия сходимости для исходной системы.");
25if (IsConvergent(source))
26{
27Console.WriteLine("Исходная система уже пригодна для итераций, преобразования не требуются.");
28return source;
29}
30
31Console.WriteLine("Исходная система не удовлетворяет достаточному условию сходимости.");
32Console.WriteLine("Подбираем эквивалентную систему линейными комбинациями уравнений.");
33Console.WriteLine();
34
35var prepared = TryCombineRows(source) ?? TryLabb(source);
36if (prepared == null)
37{
38Console.WriteLine("Привести систему к виду, удобному для итераций, не удалось.");
39return null;
40}
41
42Console.WriteLine();
43prepared.Print("Полученная система:");
44Console.WriteLine();
45if (!IsConvergent(prepared))
46{
47return null;
48}
49return prepared;
50}
51
52// Достаточное условие сходимости:  $\|C\|_1 < 1$  или  $\|C\|_\infty < 1$ 
53private static bool IsConvergent(LinearSystem system)
54{
55var size = system.Size;
56for (var row = 0; row < size; row++)
57{
58if (Math.Abs(system.GetCoefficient(row, row)) > LinearSystem.Tiny)
59{
60continue;
61}
62Console.WriteLine(" нулевой диагональный элемент a" + (row + 1) + " + (row + 1)
63+ ", разложение  $x = Cx + d$  невозможно");
64return false;
65}
66
67// Диагональные элементы матрицы перехода равны нулю, поэтому в сумму
```

```

68// входят только внедиагональные элементы столбца
69var octahedral = 0.0;
70for (var column = 0; column < size; column++)
71{
72var sum = 0.0;
73for (var row = 0; row < size; row++)
74{
75if (column == row)
76{
77continue;
78}
79sum += Math.Abs(system.GetCoefficient(row, column)) / Math.Abs(system.GetCoefficient(row, row));
80}
81octahedral = Math.Max(octahedral, sum);
82}
83
84var cubic = 0.0;
85for (var row = 0; row < size; row++)
86{
87var sum = 0.0;
88for (var column = 0; column < size; column++)
89{
90if (column == row)
91{
92continue;
93}
94sum += Math.Abs(system.GetCoefficient(row, column)) / Math.Abs(system.GetCoefficient(row, row));
95}
96cubic = Math.Max(cubic, sum);
97}
98
99Console.WriteLine(" норма матрицы, подчинённая октаэдрической: ||C||1 = "
100+ NumberFormat.Format(octahedral, Digits));
101Console.WriteLine(" норма матрицы, подчинённая кубической: ||C||∞ = "
102+ NumberFormat.Format(cubic, Digits));
103if (octahedral < 1 || cubic < 1)
104{
105Console.WriteLine(" достаточное условие сходимости выполнено, метод сходится");
106return true;
107}
108Console.WriteLine(" достаточное условие сходимости не выполнено ни по одной из норм");
109return false;
110}
111
112// Подбор линейных комбинаций уравнений с малыми целыми коэффициентами.
113// Для каждой строки перебираются комбинации и выбирается та, у которой
114// диагональный элемент преобладает над остальными. Если преобладание
115// достигнуто во всех строках, матрица невырождена и метод сходится
116private static LinearSystem? TryCombineRows(LinearSystem source)
117{
118var size = source.Size;
119if (size > MaxSizeForCombinationSearch)
120{
121Console.WriteLine("Размер системы велик для подбора комбинаций уравнений.");
122return null;
123}
124
125Console.WriteLine("Подбираем комбинации уравнений с коэффициентами от -"
126+ MaxCombinationCoefficient + " до " + MaxCombinationCoefficient + ".");
127var newRows = new double[size][];
128for (var row = 0; row < size; row++)
129{
130if (!TryFindBestRow(source, row, out var combination, out var extendedRow))
131{
132Console.WriteLine("Для строки " + (row + 1)
133+ " не нашлось комбинации уравнений с преобладающей диагональю.");
134return null;
135}
136Console.WriteLine(" уравнение " + (row + 1) + " = " + DescribeCombination(combination));

```

```

137newRows[row] = extendedRow;
138}
139return LinearSystem.FromRows(newRows);
140}
141
142private static bool TryFindBestRow(LinearSystem source, int row, out int[] combination, out double[] extendedRow)
143{
144var size = source.Size;
145combination = new int[size];
146extendedRow = new double[size + 1];
147var candidate = new int[size];
148var baseValue = 2 * MaxCombinationCoefficient + 1;
149var variants = (int)Math.Pow(baseValue, size);
150var isFound = false;
151var bestRatio = 0.0;
152var bestNonZeroCount = 0;
153var bestMaxAbs = 0;
154
155for (var index = 0; index < variants; index++)
156{
157DecodeCoefficients(index, candidate, baseValue);
158if (IsZeroCombination(candidate))
159{
160continue;
161}
162MakeFirstCoefficientPositive(candidate);
163var combined = source.GetCombinedRow(candidate);
164var ratio = LinearSystem.GetDominanceRatio(combined, row);
165if (ratio >= 1)
166{
167continue;
168}
169var nonZeroCount = CountNonZero(candidate);
170var maxAbs = MaxAbs(candidate);
171if (isFound && !IsBetter(ratio, nonZeroCount, maxAbs, bestRatio, bestNonZeroCount, bestMaxAbs))
172{
173continue;
174}
175isFound = true;
176bestRatio = ratio;
177bestNonZeroCount = nonZeroCount;
178bestMaxAbs = maxAbs;
179Array.Copy(candidate, combination, size);
180Array.Copy(combined, extendedRow, size + 1);
181}
182return isFound;
183}
184
185// Метод Лабба: прямой ход метода Гаусса даёт треугольную систему,
186// добавление к её диагонали числа delta эквивалентно линейной комбинации
187// уравнений и всегда позволяет получить преобладание диагонали
188private static LinearSystem? TryLabb(LinearSystem source)
189{
190var size = source.Size;
191var upper = new double[size, size];
192var rightParts = new double[size];
193for (var row = 0; row < size; row++)
194{
195for (var column = 0; column < size; column++)
196{
197upper[row, column] = source.GetCoefficient(row, column);
198}
199rightParts[row] = source.GetRightPart(row);
200}
201
202for (var row = 1; row < size; row++)
203{
204for (var previous = 0; previous < row; previous++)
205{

```

```

206 var factor = upper[row, previous] / upper[previous, previous];
207 upper[row, previous] = 0;
208 for (var column = previous + 1; column < size; column++)
209 {
210     upper[row, column] -= factor * upper[previous, column];
211 }
212 rightParts[row] -= factor * rightParts[previous];
213 }
214 if (Math.Abs(upper[row, row]) < LinearSystem.Tiny)
215 {
216     Console.WriteLine("Метод Лабба неприменим: система вырождена.");
217     return null;
218 }
219 }
220
221 var maxRatio = 0.0;
222 for (var row = 0; row < size; row++)
223 {
224     var extendedRow = new double[size + 1];
225     for (var column = 0; column < size; column++)
226     {
227         extendedRow[column] = upper[row, column];
228     }
229     extendedRow[size] = rightParts[row];
230     maxRatio = Math.Max(maxRatio, LinearSystem.GetDominanceRatio(extendedRow, row));
231 }
232 var delta = Math.Max(1e-6, maxRatio - 1 + 0.01);
233 Console.WriteLine("Комбинации не найдены, применяем метод Лабба:  $U + \delta \cdot D$ ,  $\delta =$ ");
234+ NumberFormat.Format(delta, Digits));
235
236 for (var row = 0; row < size; row++)
237 {
238     var factor = rightParts[row] / upper[row, row];
239     for (var column = 0; column < size; column++)
240     {
241         if (column == row)
242         {
243             upper[row, column] += delta * upper[row, column];
244         }
245     }
246     rightParts[row] += delta * factor;
247 }
248 return LinearSystem.FromRows(GetRows(upper, rightParts));
249 }
250
251 private static double[][] GetRows(double[,] coefficients, double[] rightParts)
252 {
253     var size = rightParts.Length;
254     var rows = new double[size][];
255     for (var row = 0; row < size; row++)
256     {
257         var extendedRow = new double[size + 1];
258         for (var column = 0; column < size; column++)
259         {
260             extendedRow[column] = coefficients[row, column];
261         }
262         extendedRow[size] = rightParts[row];
263         rows[row] = extendedRow;
264     }
265     return rows;
266 }
267
268 private static void DecodeCoefficients(int index, int[] candidate, int baseValue)
269 {
270     for (var i = 0; i < candidate.Length; i++)
271     {
272         candidate[i] = index % baseValue - MaxCombinationCoefficient;
273         index /= baseValue;
274     }

```

```

275}
276
277// Комбинация, отличающаяся знаком, задаёт ту же систему,
278// поэтому оставляем первый ненулевой коэффициент положительным
279private static void MakeFirstCoefficientPositive(int[] candidate)
280{
281    foreach (var coefficient in candidate)
282    {
283        if (coefficient == 0)
284        {
285            continue;
286        }
287        if (coefficient > 0)
288        {
289            return;
290        }
291        for (var i = 0; i < candidate.Length; i++)
292        {
293            candidate[i] = -candidate[i];
294        }
295        return;
296    }
297}
298
299private static bool IsZeroCombination(int[] candidate)
300{
301    foreach (var coefficient in candidate)
302    {
303        if (coefficient != 0)
304        {
305            return false;
306        }
307    }
308    return true;
309}
310
311private static int CountNonZero(int[] candidate)
312{
313    var count = 0;
314    foreach (var coefficient in candidate)
315    {
316        if (coefficient != 0)
317        {
318            count++;
319        }
320    }
321    return count;
322}
323
324private static int MaxAbs(int[] candidate)
325{
326    var maxAbs = 0;
327    foreach (var coefficient in candidate)
328    {
329        maxAbs = Math.Max(maxAbs, Math.Abs(coefficient));
330    }
331    return maxAbs;
332}
333
334// При равном отношении преобладания выбирается самая простая комбинация
335private static bool IsBetter(double ratio, int nonZeroCount, int maxAbs, double bestRatio, int bestNonZeroCount, int
    bestMaxAbs)
336{
337    if (ratio < bestRatio - 1e-12)
338    {
339        return true;
340    }
341    if (ratio > bestRatio + 1e-12)
342    {

```

```

343return false;
344}
345if (nonZeroCount != bestNonZeroCount)
346{
347return nonZeroCount < bestNonZeroCount;
348}
349return maxAbs < bestMaxAbs;
350}
351
352private static string DescribeCombination(int[] combination)
353{
354var text = new StringBuilder();
355for (var k = 0; k < combination.Length; k++)
356{
357if (combination[k] == 0)
358{
359continue;
360}
361var term = (Math.Abs(combination[k]) == 1 ? "" : Math.Abs(combination[k]) + "*") + "уравнение" + (k + 1);
362if (text.Length == 0)
363{
364text.Append(combination[k] < 0 ? "-" : "").Append(term);
365continue;
366}
367text.Append(combination[k] < 0 ? "-" : "+").Append(term);
368}
369return text.ToString();
370}
371}

```

IterationForm.cs — строка: 371

ZeidelSolver.cs

```

1using System;
2using System.Text;
3
4// Метод Зейделя. При вычислении (k+1)-го приближения xi используются уже
5// вычисленные (k+1)-ые приближения x1, ..., x(i-1), поэтому неизвестные
6// хранятся в одном одномерном массиве. Разность между приближениями
7// накапливается в скалярной переменной до перезаписи элемента
8static class ZeidelSolver
9{
10private const int TransitionDigits = 4;
11private const int SolutionDigits = 8;
12private const int MaxIterations = 100000;
13
14public static double[]? Solve(LinearSystem system, double epsilon)
15{
16var size = system.Size;
17for (var row = 0; row < size; row++)
18{
19if (Math.Abs(system.GetCoefficient(row, row)) > LinearSystem.Tiny)
20{
21continue;
22}
23Console.WriteLine("Нулевой диагональный элемент a" + (row + 1) + (row + 1)
24+ ", метод Зейделя неприменим.");
25return null;
26}
27
28// Матрица перехода и вектор свободных членов: x = Cx + d
29var matrixC = new double[size, size];
30var vectorD = new double[size];
31for (var row = 0; row < size; row++)
32{
33for (var column = 0; column < size; column++)
34{
35matrixC[row, column] = row == column

```

```

36? 0
37: -system.GetCoefficient(row, column) / system.GetCoefficient(row, row);
38}
39vectorD[row] = system.GetRightPart(row) / system.GetCoefficient(row, row);
40}
41
42Console.WriteLine("Матрица перехода C и вектор свободных членов d:");
43Console.WriteLine(NumberFormat.Block(matrixC, 0, 0, size - 1, TransitionDigits));
44Console.WriteLine("d = " + NumberFormat.Cells(vectorD, TransitionDigits));
45Console.WriteLine();
46
47// Нулевое приближение принимаем равным вектору свободных членов
48var x = new double[size];
49Array.Copy(vectorD, x, size);
50
51Console.WriteLine("Итерации метода Зейделя с точностью " + NumberFormat.Format(epsilon, 5) + ":");
52Console.WriteLine(FormatHeader(size));
53Console.WriteLine(FormatIteration(0, x, 0, false));
54
55for (var iteration = 1; iteration <= MaxIterations; iteration++)
56{
57var maxDelta = 0.0;
58for (var row = 0; row < size; row++)
59{
60var next = vectorD[row];
61for (var column = 0; column < size; column++)
62{
63if (column == row)
64{
65continue;
66}
67next += matrixC[row, column] * x[column];
68}
69maxDelta = Math.Max(maxDelta, Math.Abs(next - x[row]));
70x[row] = next;
71}
72Console.WriteLine(FormatIteration(iteration, x, maxDelta, true));
73if (maxDelta > epsilon)
74{
75continue;
76}
77Console.WriteLine("Точность достигнута за " + iteration + " итераций.");
78return x;
79}
80
81Console.WriteLine("Метод не сошёлся за " + MaxIterations + " итераций.");
82return null;
83}
84
85private static string FormatHeader(int size)
86{
87var text = new StringBuilder(" k");
88for (var i = 0; i < size; i++)
89{
90text.Append(" x").Append(i + 1);
91}
92return text.Append(" max|dx|").ToString();
93}
94
95private static string FormatIteration(int iteration, double[] x, double maxDelta, bool hasDelta)
96{
97var text = new StringBuilder(iteration.ToString().PadLeft(4));
98text.Append(' ').Append(NumberFormat.Cells(x, SolutionDigits));
99if (hasDelta)
100{
101text.Append(' ').Append(NumberFormat.Cell(maxDelta, 10));
102}
103return text.ToString();
104}

```

105}
ZeidelSolver.cs — строк: 105

NumberFormat.cs

```
1using System;
2using System.Globalization;
3
4// Форматирование чисел, строк и блоков матриц для вывода в консоль
5static class NumberFormat
6{
7    private const int CellWidth = 11;
8
9    public static string Format(double value, int digits)
10{
11    return value.ToString("F" + digits, CultureInfo.CurrentCulture);
12}
13
14    public static string Cell(double value, int digits)
15{
16    return Format(value, digits).PadLeft(CellWidth);
17}
18
19    public static string Cells(double[] values, int digits)
20{
21    var cells = new string[values.Length];
22    for (var i = 0; i < values.Length; i++)
23    {
24        cells[i] = Cell(values[i], digits);
25    }
26    return string.Join(" ", cells);
27}
28
29// Печатает прямоугольный блок матрицы: строки от firstRow,
30// столбцы от firstColumn до lastColumn включительно
31    public static string Block(double[,] matrix, int firstRow, int firstColumn, int lastColumn, int digits)
32{
33    var lines = new string[matrix.GetLength(0) - firstRow];
34    for (var row = firstRow; row < matrix.GetLength(0); row++)
35    {
36        var cells = new string[lastColumn - firstColumn + 1];
37        for (var column = firstColumn; column <= lastColumn; column++)
38        {
39            cells[column - firstColumn] = Cell(matrix[row, column], digits);
40        }
41        lines[row - firstRow] = string.Join(" ", cells);
42    }
43    return string.Join(Environment.NewLine, lines);
44}
45}
```

NumberFormat.cs — строк: 45

Всего строк: 982.